19.1 Razor Pages vs. MVC in ASP.NET Core

In this book I focus on Razor Pages, but I have also mentioned that Razor Pages use the ASP.NET Core MVC framework behind the scenes and that you can choose to use the MVC framework directly if you wish. Additionally, if you're creating an API for working with mobile or client-side apps, and you don't want to (or can't) use minimal APIs, you may well use the MVC framework directly by creating web API controllers.

NOTE I look at how to build web APIs with the MVC framework in chapter 20.

So what are the differences between Razor Pages and the MVC framework, and when should you choose one or the other?

If you're new to ASP.NET Core, the answer is pretty simple: use Razor Pages for server-side rendered applications, and use minimal APIs (or web API controllers) for building APIs. There are nuances to this advice, which I discuss in section 19.5, but that distinction will serve you well for now.

Naming is hard, again

Microsoft have a long history of creating a framework and naming it after a generic concept: MVC, Web Forms, Web Pages, Multi-platform App UI, and so on. It's frankly incredible that Blazor survived! Web API is no different.

In legacy ASP.NET, Microsoft created a web API framework, which was similar in *design* to the existing MVC framework, but also was not interoperable. You therefore had *MVC controllers*, which were controller classes used with the MVC framework to generate HTML, and *web API controllers*, which were controller classes used with the web API framework, to generate JavaScript Object Notation (JSON) or Extensible Markup Language (XML).